Análisis de Selección del RDBMS

Sistema de Gestión de Producción y Venta de Pollos de Engorde

Bogotá D.C., noviembre de 2026

Versión 1.2

Gabriela Poveda, Dylan Mancipe y Juan Diego López
Universidad El Bosque · Bases de Datos 2
Docente: Ing., Esp., M.Ed. Christian Felipe Duarte

TABLA DE CONTENIDO

Historial de versiones 3
1. Introducción 3
2. Método 3
3. Criterios y pesos 3
4. Matriz de decisión 4
5. Justificación por criterio 4
6. Análisis de sensibilidad 5
7. Versión instalada 5
8. Cómo se aprovechó cada criterio en la implementación 5
9. Seguridad e integridad 6
10. Conclusión 7
Referencias 7

Historial de versiones

Fecha	Versión	Autor	Descripción
05/11/2026	1.0	Gabriela Poveda	Primera versión: análisis del acta ampliado con la versión instalada y la evidencia de la implementación.
10/11/2026	1.1	Gabriela Poveda	Revisión del equipo
11/11/2026	1.2	Juan Diego López	Correcciones de consistencia con la implementación.

1. Introducción

Este documento presenta el análisis de selección del sistema gestor de bases de datos relacional (RDBMS), entregable de la Fase 2. La decisión se tomó en el acta de constitución del 22/10/2026 con una matriz de decisión ponderada; aquí se presenta como documento propio y se amplía con la versión instalada (PostgreSQL 18) y con la forma en que cada criterio se aprovechó en la implementación.

El enunciado permite MySQL, PostgreSQL, MariaDB o SQL Server, y excluye Oracle. El modelo relacional sigue los principios de Codd [8] y la normalización hasta 3FN descrita por Elmasri y Navathe [9].

2. Método

Se aplicó un análisis multicriterio por suma ponderada, con pesos asignados según la importancia de cada criterio para los objetivos del proyecto, en línea con los métodos de decisión multicriterio de Saaty [1] y Triantaphyllou [2]. Cada alternativa se calificó de 1 (muy deficiente) a 5 (excelente) y el puntaje final es la suma de cada calificación por su peso. Los pesos más altos corresponden a la lógica procedural y al control transaccional, porque equivalen al 20 % de la rúbrica (procedimientos, funciones, triggers, transacciones, excepciones, seguridad e integridad).

3. Criterios y pesos

Cód.	Criterio	Qué evalúa	Peso
C1	Lógica procedural	Procedimientos, funciones y tipos de triggers disponibles	25 %
C2	Transacciones y excepciones	COMMIT/ROLLBACK en procedimientos, manejo de errores, DDL transaccional	20 %
C3	Integridad declarativa	CHECK, dominios, tipos enumerados e índices parciales	15 %
C4	Cercanía a PL/SQL	Transferencia de lo aprendido en clase con Oracle	10 %

Cód.	Criterio	Qué evalúa	Peso
C5	Licencia y costo	Gratuidad y restricciones de uso	10 %
C6	Despliegue en VM Linux	Soporte en Ubuntu Server y consumo de recursos	10 %
C7	Integración con Java (Spring Boot)	Madurez del controlador JDBC y llamado a procedimientos	5%
C8	Herramientas de modelado	Soporte en Data Modeler o Workbench, exigidos en la Fase 1	5%

Tabla 1. Criterios de selección.

4. Matriz de decisión

Criterio (peso)	PostgreSQL	MySQL	MariaDB	SQL Server
C1 Lógica procedural (25 %)	5	3	3	4
C2 Transacciones y excepciones (20 %)	5	3	3	4
C3 Integridad declarativa (15 %)	5	3	3	4
C4 Cercanía a PL/SQL (10 %)	5	3	4	2
C5 Licencia y costo (10 %)	5	4	5	3
C6 Despliegue en VM Linux (10 %)	5	5	5	3
C7 Integración con Java (5 %)	5	5	5	5
C8 Herramientas de modelado (5 %)	2	5	4	4
Puntaje ponderado	**4,85**	**3,50**	**3,65**	**3,65**

Tabla 2. Matriz de decisión ponderada (acta de constitución).

5. Justificación por criterio

- **C1. Lógica procedural:** PL/pgSQL ofrece triggers BEFORE, AFTER e INSTEAD OF, a nivel de fila y de sentencia [3]. En MySQL y MariaDB los triggers solo son por fila, no admiten COMMIT ni ROLLBACK y no pueden modificar la tabla que los disparó [4].
- **C2. Transacciones y excepciones:** Desde la versión 11, los PROCEDURE de PostgreSQL ejecutan COMMIT y ROLLBACK; los bloques EXCEPTION actúan como puntos de guardado implícitos y el DDL es transaccional [3]. En MySQL una sentencia DDL provoca un commit implícito.
- **C3. Integridad declarativa:** PostgreSQL tiene índices parciales, tipos enumerados (ENUM), dominios y columnas generadas. MySQL no tiene índices parciales.
- **C4. Cercanía a PL/SQL:** PL/pgSQL está inspirado en PL/SQL de Oracle, el lenguaje usado en el curso, y reutiliza la estructura de bloques, cursores y excepciones. T-SQL de SQL Server difiere más.

- **C5. Licencia:** PostgreSQL se distribuye con la PostgreSQL License, libre y permisiva. SQL Server 2025 Express es gratuito pero limita el tamaño de cada base, la memoria y los núcleos; la edición Developer solo se licencia para desarrollo y pruebas [5].
- **C6 y C7. Despliegue e integración:** PostgreSQL tiene paquetes oficiales para Ubuntu Server y corre con pocos recursos en una VM. Su controlador JDBC permite invocar procedimientos con CALL y funciones con SELECT desde Spring Boot. Los cuatro motores tienen controladores JDBC maduros, por lo que C7 no los diferencia.
- **C8. Modelado (desventaja):** Oracle SQL Developer Data Modeler no genera DDL nativo para PostgreSQL. Mitigación: el DDL se escribió a mano y se versionó en los scripts sql/; los modelos Crow's Foot se generaron en Data Modeler importando una versión equivalente del DDL en sintaxis Oracle, por lo que coinciden con la base real.

6. Análisis de sensibilidad

Escenario	PostgreSQL	MySQL	MariaDB	SQL Server
Base (pesos de la Tabla 1)	4,85	3,50	3,65	3,65
Pesos iguales (12,5 % cada criterio)	4,63	3,88	4,00	3,63
Modelado reforzado (C1 15 %, C8 15 %)	4,55	3,70	3,75	3,65

Tabla 3. Sensibilidad de la decisión (acta de constitución).

PostgreSQL conserva el primer lugar en todos los escenarios. Incluso en el que más favorece a las herramientas de modelado, su ventaja en lógica procedural, transacciones e integridad compensa su única desventaja.

7. Versión instalada

Se instaló PostgreSQL 18 en una VM con Ubuntu 26.04 LTS Desktop en instalación mínima, por las razones del análisis de selección del SO. La base pollos_hermanos se crea con los scripts del repositorio en este orden: modulo_00_base_y_roles.sql, propuesta_seguridad.sql, modulo_01 a modulo_12 y correccion_02_zona_horaria.sql. Cada script corre dentro de una transacción (BEGIN … COMMIT), de modo que un error deja la base como estaba. PostgreSQL desciende del proyecto POSTGRES de Stonebraker y Rowe, que introdujo la extensibilidad de tipos y reglas que hoy aprovechan los ENUM y triggers del proyecto [6].

8. Cómo se aprovechó cada criterio en la implementación

Criterio	Evidencia en el proyecto
C1 Lógica procedural	20 procedimientos (sp_), 48 funciones (fn_) y 25 triggers declarados en los scripts. Triggers BEFORE de validación (tgr_biu_lote_validar), AFTER de efecto (tgr_ai_pago_venta, que actualiza el estado de la venta) y de

Criterio	Evidencia en el proyecto
	sentencia (tgr_bt_auditoria_no_truncar, BEFORE TRUNCATE). Además, fn_auditar_tablas crea con SQL dinámico un trigger tgr_aiud_<tabla>_auditoria en cada tabla.
C2 Transacciones y excepciones	Cada procedimiento hace su trabajo en un bloque interno con EXCEPTION, que traduce el error a español con fn_mensaje_restriccion, y fuera del bloque decide COMMIT o ROLLBACK (por ejemplo, sp_registrar_lote). El DDL transaccional permitió aplicar el cambio de producción escalonada (DROP INDEX, ALTER TABLE y ajuste de datos) en un solo bloque atómico (módulo 07). Las pruebas SQL corren en una transacción que se deshace al final [7].
C3 Integridad declarativa	Seis tipos ENUM (estado_galpon, estado_lote, estado_pedido, estado_venta, estado_entrega, tipo_producto_sanitario); restricciones CHECK con nombre, incluidas expresiones regulares (ck_lote_codigo); columna generada venta.valor_total; índices parciales ix_lote_activo_por_galpon (la regla original de un lote activo por galpón, sin código), su reemplazo ix_lote_galpon_activo e ix_venta_cartera. No se usaron DOMAIN: los ENUM y CHECK cubrieron los casos.
C4 Cercanía a PL/SQL	Se reutilizó la nomenclatura del curso: variables v_/vv_/vd_, registros r_, parámetros p_, cursores c_, excepciones e_, funciones fn_, procedimientos sp_, índices ix_ y triggers tgr_<b|a><i|u|d|t>_….
C5 Licencia	Costo de licencia $0, según el presupuesto del acta.
C6 Despliegue	PostgreSQL 18 desde los paquetes oficiales de Ubuntu 26.04 LTS; acceso restringido con ufw y pg_hba.conf al equipo del backend.
C7 Integración	El backend usa JdbcTemplate con CALL sp_… y SELECT … FROM v_…/fn_…, y Spring Data JPA con ddl-auto=validate: Hibernate solo verifica el esquema, nunca lo crea.
C8 Modelado	El DDL de PostgreSQL se escribió a mano y está versionado en sql/; los modelos Crow's Foot (E-R y relacional) se generaron en Data Modeler importando una versión equivalente del DDL en sintaxis Oracle, por lo que coinciden con la base real.

Tabla 4. Evidencia de cada criterio en la implementación.

9. Seguridad e integridad

- **Privilegios mínimos:** Los objetos pertenecen a pollos_owner, un rol sin login. La aplicación usa pollos_app, que solo tiene DML y ejecución; cada módulo le revoca todo y le concede solo lo necesario. Por ejemplo, las compras y los pagos no se pueden borrar.
- **Auditoría inmutable:** La tabla auditoria guarda en JSONB la fila antes y después de cada cambio, el usuario de la aplicación (enviado por el backend con set_config), el rol de base de datos y la transacción. Un trigger impide actualizarla, borrarla o truncarla; la función

que escribe en ella es SECURITY DEFINER para que pollos_app audite sin poder modificarla.

- **Mensajes controlados:** La tabla mensaje_restriccion asocia cada restricción con un mensaje en español, que la aplicación muestra al usuario.

10. Conclusión

PostgreSQL fue la mejor opción según el análisis (4,85 de 5) y la implementación lo confirmó: la lógica del negocio quedó en la base de datos con procedimientos, funciones y triggers, cada operación es atómica y la seguridad se resolvió con roles de privilegios mínimos y auditoría inmutable. La única desventaja prevista, el modelado en Data Modeler, se mitigó escribiendo el DDL a mano y generando los modelos en Data Modeler a partir de una versión equivalente en sintaxis Oracle.

Referencias

[1] T. L. Saaty, "How to make a decision: The analytic hierarchy process," *European Journal of Operational Research*, vol. 48, no. 1, pp. 9-26, 1990, doi: 10.1016/0377-2217(90)90057-I.

[2] E. Triantaphyllou, *Multi-Criteria Decision Making Methods: A Comparative Study*. Boston, MA, EE. UU.: Springer, 2000, doi: 10.1007/978-1-4757-3157-6.

[3] PostgreSQL Global Development Group, *PostgreSQL 18 Documentation*, 2026. [En línea]. Disponible en: https://www.postgresql.org/docs/18/. [Consultado: 10-nov-2026].

[4] Oracle Corporation, "Stored objects," en *MySQL 9.0 Reference Manual*, 2024. [En línea]. Disponible en: https://dev.mysql.com/doc/refman/9.0/en/stored-objects.html. [Consultado: 22-oct-2026].

[5] Microsoft, "Editions and components of SQL Server 2025," *Microsoft Learn*, 2025. [En línea]. Disponible en: https://learn.microsoft.com/en-us/sql/sql-server/editions-and-components-of-sql-server-2025. [Consultado: 22-oct-2026].

[6] M. Stonebraker y L. A. Rowe, "The design of POSTGRES," en *Proc. ACM SIGMOD Int. Conf. Management of Data*, Washington, D.C., EE. UU., 1986, pp. 340-355.

[7] T. Haerder y A. Reuter, "Principles of transaction-oriented database recovery," *ACM Computing Surveys*, vol. 15, no. 4, pp. 287-317, 1983.

[8] E. F. Codd, "A relational model of data for large shared data banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377-387, 1970, doi: 10.1145/362384.362685.

[9] R. Elmasri y S. B. Navathe, *Fundamentals of Database Systems*, 7.ª ed. Hoboken, NJ, EE. UU.: Pearson, 2016.